Builder Pattern Explanation

builder pattern

Sure! Let's break down the **Builder Pattern** in Java clearly, step by step, with examples and when to use it.

---

1. What is the Builder Pattern?

The **Builder Pattern** is a **creational design pattern** used to construct complex objects step by step. It separates the construction of a complex object from its representation so that the same construction process can create different representations.

Key idea: Instead of having a constructor with many parameters (which is hard to read and maintain), the builder pattern allows you to build the object piece by piece.

---

2. Why use the Builder Pattern?

- To avoid **telescoping constructors** (constructors with many parameters).
- To improve **readability** and **maintainability** of object creation.
- To make the object **immutable** while still allowing flexible initialization.

---

3. Real-world Example

Think about ordering a **pizza**. A pizza can have:

- Size (small, medium, large)
- Cheese
- Pepperoni
- Olives
- Extra toppings

If we had a constructor, it might look like:

```
Pizza pizza = new Pizza("Large", true, true, false, true);
```

It’s hard to know which boolean is which!

Instead, the **Builder Pattern** makes it readable:

```
Pizza pizza = new PizzaBuilder()
                  .setSize("Large")
                  .addCheese()
                  .addPepperoni()
                  .addExtraToppings()
                  .build();
```

---

4. Java Implementation

Step 1: Create the Product Class

```
public class Pizza {
    private String size;
    private boolean cheese;
    private boolean pepperoni;
    private boolean olives;

    // Private constructor
    private Pizza(PizzaBuilder builder) {
        this.size = builder.size;
        this.cheese = builder.cheese;
        this.pepperoni = builder.pepperoni;
        this.olives = builder.olives;
    }

    @Override
    public String toString() {
        return "Pizza [size=" + size + ", cheese=" + cheese +
               ", pepperoni=" + pepperoni + ", olives=" + olives + "]";
    }

    // Static Builder Class
    public static class PizzaBuilder {
        private String size;
        private boolean cheese;
        private boolean pepperoni;
        private boolean olives;

        public PizzaBuilder setSize(String size) {
            this.size = size;
            return this;  // returns the builder itself for chaining
        }

        public PizzaBuilder addCheese() {
            this.cheese = true;
            return this;
        }

        public PizzaBuilder addPepperoni() {
            this.pepperoni = true;
            return this;
        }

        public PizzaBuilder addOlives() {
            this.olives = true;
            return this;
        }

        public Pizza build() {
            return new Pizza(this); // constructs the Pizza
        }
    }
}
```

Step 2: Use the Builder

```
public class Main {
    public static void main(String[] args) {
        Pizza pizza = new Pizza.PizzaBuilder()
                            .setSize("Large")
                            .addCheese()
                            .addPepperoni()
                            .build();

        System.out.println(pizza);
    }
}
```

Output:

```
Pizza [size=Large, cheese=true, pepperoni=true, olives=false]
```

✅ Clear, readable, and maintainable.

---

5. Key Points

1. **Chaining:** Builder methods return `this` to allow method chaining.
2. **Immutable Product:** The main object (`Pizza`) can have a private constructor, so it's immutable once built.
3. **Optional Fields:** You can set only the fields you want without writing multiple constructors.
4. **Setter in Builder:** You **must** have some way to set values in the builder (like setter methods). But these setters are in the **builder**, not in the product class.

---

6. When to Use Builder Pattern

- When a class has **many optional parameters**.
- When creating **immutable objects** with flexible initialization.
- When you want **readable and maintainable code** instead of long constructors.

If you want, I can also show a **real-world Java example using the Builder Pattern with Lombok** where you don't even have to manually write the builder methods—it's super concise!

Do you want me to show that?

> It separates the construction of a complex object from its representation so that the same construction process can create different representations. ye line smaj nhi aaye

This line is from the definition of the **Builder Pattern**, but it sounds confusing. Let's simplify it. 😊

Original line:

> *It separates the construction of a complex object from its representation so that the same construction process can create different representations.*

1️⃣ "Separates the construction of a complex object"

This means **object banane ka process alag kar diya jata hai**.

Normally:

```
House house = new House(4, 2, true, true);
```

Yaha object **direct constructor se ban raha hai**.

But Builder Pattern me:

```
House house = new House.Builder()
                    .setDoors(4)
                    .setWindows(2)
                    .setGarage(true)
                    .build();
```

Yaha **object banane ka logic Builder class me shift ho gaya**.

So:

Construction logic → Builder class
Final object → House class

2️⃣ "From its representation"

Representation ka matlab hai **final object kaisa dikhega**.

Example:

House ke types ho sakte hain:

- Simple house
- Luxury house
- Villa

Same class se different houses ban sakte hain.

Example:

```
House simpleHouse = new House.Builder()
                        .setDoors(2)
                        .setWindows(2)
                        .build();

House luxuryHouse = new House.Builder()
                        .setDoors(6)
                        .setWindows(10)
                        .setGarage(true)
                        .build();
```

Yaha **final representation alag hai**.

3️⃣ "Same construction process"

Builder ka **same process use ho raha hai**:

```
Builder → set values → build()
```

But output object different ho sakta hai.

Example:

```
Builder → small house
Builder → luxury house
Builder → villa
```

4️⃣ Simple Real Life Example 🏠

Think about **making a burger** 🍔

Steps always same:

```
1 add bun
2 add patty
3 add toppings
4 build burger
```

But result different ho sakta hai:

- Veg burger
- Cheese burger
- Double patty burger

Process same → Result different

This is exactly what that line means.

✅ **In simple words:**

Builder Pattern me:

- **Object banane ka process alag class me hota hai**
- **Same process se different objects ban sakte hain**

If you want, I can also show **the biggest problem builder pattern solves (Telescoping Constructor Problem)** which is a **very common Java interview question**. 🚀

```
// Private constructor
    private Pizza(PizzaBuilder builder) {
        this.size = builder.size;
        this.cheese = builder.cheese;
        this.pepperoni = builder.pepperoni;
        this.olives = builder.olives;
    }  this.size=builder.size kya hai ye
```

Good question 👍
`this.size = builder.size` samajhna **Builder Pattern ka core** hai.

Let's break it.

1️⃣ `this.size` kya hai?

`this.size` **Pizza object ka variable** hai.

```
public class Pizza {

    private String size;
```

Yani **Pizza class ka field**.

2️⃣ `builder.size` kya hai?

`builder.size` **Builder class ka variable** hai.

```
public static class PizzaBuilder {

    private String size;
```

Yani jab user builder use karta hai:

```
Pizza pizza = new Pizza.PizzaBuilder()
                .setSize("Large")
                .build();
```

To `PizzaBuilder` object ke andar value store hoti hai:

```
builder.size = "Large"
```

3️⃣ Constructor me kya ho raha hai?

```
private Pizza(PizzaBuilder builder) {
    this.size = builder.size;
}
```

Yaha **Builder se value copy ho rahi hai Pizza object me**.

Step by step:

Step 1: Builder create

```
PizzaBuilder builder = new PizzaBuilder();
```

Step 2: Value set

```
builder.setSize("Large");
```

Ab:

```
builder.size = "Large"
```

Step 3: build() call

```
return new Pizza(this);
```

Yaha builder **Pizza constructor ko pass ho raha hai**.

Step 4: Constructor execute

```
this.size = builder.size;
```

Matlab:

```
Pizza.size = "Large"
```

4️⃣ Simple analogy 🧠

Think:

Builder = Form

```
size = Large
cheese = true
pepperoni = true
```

Then final object create hota hai:

```
Pizza pizza = new Pizza(builder)
```

Constructor bolta hai:

```
Pizza.size = builder.size
Pizza.cheese = builder.cheese
Pizza.pepperoni = builder.pepperoni
```

Builder se **data copy karke final object banaya ja raha hai**.

5️⃣ Visual Flow

```
User
 ↓
PizzaBuilder
 ↓ setSize("Large")
builder.size = "Large"
 ↓
build()
 ↓
new Pizza(builder)
 ↓
this.size = builder.size
 ↓
Pizza object created
```

✅ Simple sentence

`this.size = builder.size` means:

> Builder object me jo value stored hai, usko final Pizza object me copy karo.

If you want, I can also show **a very tricky interview question about Builder Pattern** that many senior Java developers get wrong. 🚀

If you want, I can also show the biggest problem builder pattern solves (Telescoping Constructor Problem) which is a very common Java interview question. 🚀

Yes 👍 The **biggest problem Builder Pattern solves** is called the **Telescoping Constructor Problem**.

Let's understand it clearly.

1️⃣ What is Telescoping Constructor Problem?

When a class has **many optional parameters**, we create **multiple constructors**.

Example:

```
public class Pizza {

    private String size;
    private boolean cheese;
    private boolean pepperoni;
    private boolean olives;

    public Pizza(String size) {
        this(size, false);
    }

    public Pizza(String size, boolean cheese) {
        this(size, cheese, false);
    }

    public Pizza(String size, boolean cheese, boolean pepperoni) {
        this(size, cheese, pepperoni, false);
    }

    public Pizza(String size, boolean cheese, boolean pepperoni, boolean olives) {
        this.size = size;
        this.cheese = cheese;
        this.pepperoni = pepperoni;
        this.olives = olives;
    }
}
```

Now creating objects:

```
Pizza p1 = new Pizza("Large");
Pizza p2 = new Pizza("Large", true);
Pizza p3 = new Pizza("Large", true, true);
Pizza p4 = new Pizza("Large", true, true, true);
```

This is called **Telescoping Constructors**.

Why?

Because constructors **keep expanding like a telescope** 🔭.

2️⃣ Problems with Telescoping Constructors

❌ Hard to read

```
Pizza pizza = new Pizza("Large", true, false, true);
```

You cannot easily tell:

```
true → cheese?
false → pepperoni?
true → olives?
```

❌ Too many constructors

If there are **10 fields**, combinations become huge.

Example:

```
Pizza()
Pizza(size)
Pizza(size, cheese)
Pizza(size, cheese, pepperoni)
Pizza(size, cheese, pepperoni, olives)
...
```

Class becomes messy.

❌ Maintenance problem

If a new field is added:

```
boolean mushroom
```

You must change **all constructors**.

3️⃣ Builder Pattern Solution

Builder Pattern removes this problem.

```
Pizza pizza = new Pizza.PizzaBuilder()
                    .setSize("Large")
                    .addCheese()
                    .addOlives()
                    .build();
```

Advantages:

✔️ Very readable
✔️ Optional parameters easy
✔️ No constructor explosion
✔️ Easy to maintain

4️⃣ Real Industry Example

In **Spring Boot**, many classes use Builder Pattern.

Example conceptually:

```
HttpRequest request = HttpRequest.newBuilder()
                      .uri(url)
                      .header("Authorization", token)
                      .GET()
                      .build();
```

Here also builder constructs the object step by step.

5️⃣ Short Interview Answer

If interviewer asks:

What problem does Builder Pattern solve?

You can say:

> Builder Pattern solves the **Telescoping Constructor Problem**, where a class has many optional parameters leading to multiple confusing constructors.

✅ If you want, I can also show a **very famous Builder Pattern example from Effective Java (Joshua Bloch)** that is asked in **Amazon / Google Java interviews**.

yes

Great 👍 Let's see the **famous Builder Pattern example from the book *Effective Java* by Joshua Bloch**. This example is **very commonly asked in Java interviews (Amazon, Google, etc.)**.

1️⃣ Problem: Class with Many Parameters

Suppose we have a **NutritionFacts** class for food items.

Fields:

- servingSize (required)
- servings (required)
- calories (optional)
- fat (optional)
- sodium (optional)
- carbohydrate (optional)

2️⃣ Bad Solution (Telescoping Constructors)

```java
public class NutritionFacts {

    private final int servingSize;
    private final int servings;
    private final int calories;
    private final int fat;
    private final int sodium;
    private final int carbohydrate;

    public NutritionFacts(int servingSize, int servings) {
        this(servingSize, servings, 0);
    }

    public NutritionFacts(int servingSize, int servings, int calories) {
        this(servingSize, servings, calories, 0);
    }

    public NutritionFacts(int servingSize, int servings, int calories, int fat) {
        this(servingSize, servings, calories, fat, 0);
    }

    public NutritionFacts(int servingSize, int servings, int calories, int fat, int sodium) {
        this(servingSize, servings, calories, fat, sodium, 0);
    }

    public NutritionFacts(int servingSize, int servings, int calories, int fat, int sodium, int carbol
        this.servingSize = servingSize;
        this.servings = servings;
        this.calories = calories;
        this.fat = fat;
        this.sodium = sodium;
        this.carbohydrate = carbohydrate;
    }
}
```

Creating object:

```java
NutritionFacts coke = new NutritionFacts(240, 8, 100, 0, 35, 27);
```

Problem ❌

You don't know what **100, 0, 35, 27** mean.

3️⃣ Builder Pattern Solution

```java
public class NutritionFacts {

    private final int servingSize;
    private final int servings;
    private final int calories;
    private final int fat;
    private final int sodium;
    private final int carbohydrate;

    public static class Builder {

        // required parameters
        private final int servingSize;
        private final int servings;

        // optional parameters
        private int calories = 0;
        private int fat = 0;
        private int sodium = 0;
        private int carbohydrate = 0;

        public Builder(int servingSize, int servings) {
            this.servingSize = servingSize;
            this.servings = servings;
        }

        public Builder calories(int val) {
            calories = val;
            return this;
        }

        public Builder fat(int val) {
            fat = val;
            return this;
        }

        public Builder sodium(int val) {
            sodium = val;
            return this;
        }

        public Builder carbohydrate(int val) {
            carbohydrate = val;
            return this;
        }

        public NutritionFacts build() {
            return new NutritionFacts(this);
        }
    }

    private NutritionFacts(Builder builder) {
        servingSize = builder.servingSize;
        servings = builder.servings;
        calories = builder.calories;
        fat = builder.fat;
        sodium = builder.sodium;
        carbohydrate = builder.carbohydrate;
    }
}
```

4️⃣ Object Creation (Very Readable)

```java
NutritionFacts coke = new NutritionFacts.Builder(240, 8)
                        .calories(100)
                        .sodium(35)
                        .carbohydrate(27)
                        .build();
```

Now it is **very clear**:

```
servingSize = 240
servings = 8
calories = 100
sodium = 35
carbohydrate = 27
```

5️⃣ Why This Example is Famous

This pattern became popular because:

- It keeps **objects immutable**
- It supports **optional parameters**
- Code becomes **very readable**
- Used widely in **Java libraries**

6️⃣ Where You See This in Real Java APIs

Examples in real Java:

```
StringBuilder sb = new StringBuilder();

Stream.builder()

HttpRequest.newBuilder()
```

Many frameworks also use it (Spring, Lombok, etc.).

✅ Interview One-Line Answer

> Builder Pattern is used to create complex objects step by step and solves the **Telescoping Constructor Problem**, making object creation more readable and flexible.

If you want, I can also show **3 extremely tricky Builder Pattern interview questions** that even **5+ year Java developers often fail**. 🚀